AutoHeal Code Execution and Self Correction

Generative AI Laboratory Project Report

Course 216U42E734 | Semester VII | Academic year 2026–27

16014223003 | Aayush Kumar Tiwari | LY AI & DS B2
16014223061 | Prateeksha Kini | LY AI & DS B3

Joint submission | 16 October 2026 | Version 2.0.0

https://github.com/aayushtiwari845/AutoHeal

Implementation revision e5a064a7f31a. The report records this source snapshot and the evidence identified in its final section.

Abstract

AutoHeal is an end-to-end Generative AI application that turns a natural-language task into Python, executes the candidate, evaluates the output and uses failure evidence to request a repair. The project addresses a practical weakness of code generation: plausible source and a successful process exit can still produce a wrong answer. An independent oracle therefore checks the three fixed lab scenarios, while custom output is explicitly labelled as structurally checked.

The application combines a Win95-inspired browser interface with a local Python service. Automatic flow demonstrates uninterrupted orchestration; manual flow exposes the phase boundaries so a supervisor can inspect the candidate, exception, feedback and revised program. Local Ollama is the default provider, with an optional configured API path. Every run is bounded by an attempt limit and each program by an execution timeout.

This report separates live generation, declared controlled faults and scripted replay. It records the current implementation and real verification evidence. The executor supports trusted local demonstrations and is not a hard operating-system sandbox.

Theoretical foundation

ReAct interleaves model proposals, actions and environment observations [1]. Here the action is a Python subprocess and the observation is its execution record. Reflexion uses verbal feedback to improve subsequent attempts without updating model weights [2]. AutoHeal applies this idea through the latest candidate, a concise diagnosis and bounded failure evidence. This is an engineering application inspired by these frameworks, rather than a reproduction of their complete agents or a new training method.

Each repair is executed and independently checked. A displayed explanation is not proof of correctness, and a bounded loop can exhaust its budget. Temperature zero does not guarantee identical results across providers or machines; private hidden reasoning is neither required nor treated as evidence.

Problem Objectives and Assessment Scope

Problem statement

A one-shot coding assistant cannot observe the real consequence of its own proposal unless an execution system supplies that evidence. Generated Python may fail to parse, call an unavailable method, mishandle data or satisfy the syntax while violating the task. Copying an error into a new prompt by hand is slow and makes the sequence difficult to assess. The lab task therefore requires a repeatable execution and repair loop with explicit state and termination.

Project objectives

We implement generation, execution, evaluation and reflection as observable phases. We capture stdout, stderr, return code and the generated error line; convert failures into concise structured feedback; and stop immediately after validated success. We enforce at most five attempts and a maximum five-second execution budget. The interface makes the same underlying process demonstrable through automatic and manually advanced control.

Assessment criterion	Marks	Concrete evidence
Loop logic and state machine	3	Explicit phases, attempt records, captured streams and return code, success and exhaustion stops
Context and error reflection	3	Exception type and line, bounded traceback, failing JSON field paths and task-rule reminders
Code execution environment	2	Fresh constrained subprocess, timeout, bounded capture and reported Windows memory control
Viva and failure analysis	2	Observed traces, logical error rejection, non-convergence analysis and reproducible demonstration

Scope and success criteria

The intended outcome is a complete and explainable lab project, not a claim of universal autonomous programming. A preset succeeds only when valid output matches its independent contract. A custom task may establish successful execution and JSON structure without establishing the answer's domain correctness. A failed repair can still demonstrate correct loop behavior if its evidence, retries and terminal status are recorded honestly.

The course assessment assigns ten marks to this execution and auto-heal component. Its broader mini-project context is the design of an end-to-end Generative AI application. The separate model-card assignment is outside this report's implementation scope.

Requirements and Reproducible Setup

Requirement	Implementation choice	Reason
Application runtime	Python standard library and a modern browser	Portable backend with no web framework installation
Local model	Ollama and qwen2.5-coder:7b	Code-specialized local inference without a hosted key
Interface	HTML, CSS and JavaScript	Retro desktop appearance with inspectable source
Execution	Separate Python child for each candidate	Contain errors and enforce execution limits
Optional inference	Configured compatible API transport	Permit provider comparison when explicitly enabled
Version control	Dedicated Git repository	Source, tests, screenshots, report and evidence stay reviewable

Local launch sequence

```
git clone https://github.com/aayushtiwari845/AutoHeal.git
cd AutoHeal
ollama pull qwen2.5-coder:7b
python launcher.py
```

The launcher starts or connects to the local service and opens the dashboard. A first model download requires internet access and sufficient disk space; subsequent local runs use the installed model. Model loading and generation may take much longer than a five-second code execution. The program timeout applies to the generated child process, not to the complete model request.

Environment recorded for evaluation

Python 3.12.14. Windows laptop; Intel Core i7-13650HX; 23.71 GiB installed usable RAM reported by Windows; NVIDIA GeForce RTX 3050 6GB Laptop GPU. These describe the observed environment rather than a minimum specification or a throughput guarantee. The quantized 7B distribution is approximately 4.7GB on disk; runtime memory also depends on context, loading and offloading [11].

Optional API configuration

API use is optional. The server reads AUTOHEAL_API_BASE, AUTOHEAL_API_MODEL and AUTOHEAL_API_KEY, with GROQ_API_KEY accepted as a credential fallback. The default compatible base is https://api.groq.com/openai/v1. An API model must be configured explicitly. A local .env file is supported and excluded from Git; an existing environment value takes precedence. No credential belongs in a run request, screenshot, report or generated-child environment.

For the optional route, copy .env.example to .env, set the documented values locally and restart the service. The transport requires an HTTPS base, refuses redirects and uses the configured model. The public provider catalog reports availability and model identity without exposing the key. Local Ollama and scripted replay require no key.

System Architecture and Data Flow

The browser presents controls and observations while the local Python service owns state, provider calls and execution. Generated Python never runs inside the browser or the orchestration process. The model provider proposes code, the child process produces execution evidence, and the independent oracle decides whether a preset's answer satisfies the task.

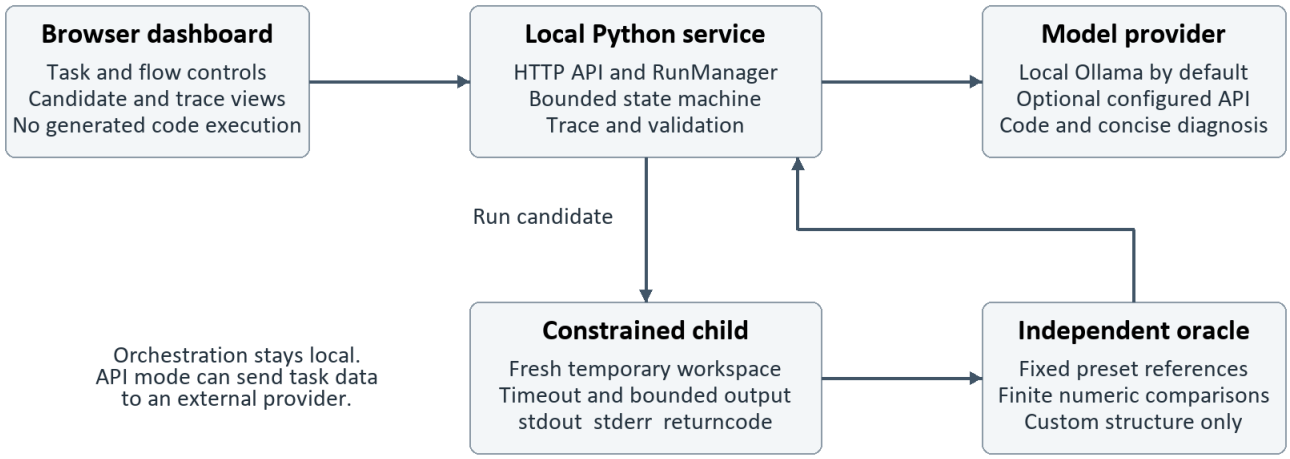


Figure 1 AutoHeal component and data flow diagram derived from the implementation

Request to result

A user selects a task, provider, flow and attempt budget. The API validates the configuration and creates a run. The manager sends the task and DATA to the provider, records the candidate and launches the constrained executor. The resulting output is parsed and checked. A successful decision ends the run; a failure becomes bounded repair context for the next candidate.

Why the boundaries are separate

Separating generation, execution and evaluation prevents a provider from declaring its own proposal correct. Separating the Python child from the server prevents an ordinary runtime exception from crashing the interface. Keeping the reference validator outside generated source also lets the project reject a plausible but incorrect JSON object after exit code zero.

Local inference keeps task content on the local machine when the selected provider is Ollama. Optional API inference changes that data path: the task, data, previous code and bounded failure evidence can be transmitted to the configured service. The deployment therefore makes provider choice visible instead of applying a blanket local privacy claim.

Automatic and Manual State Machine

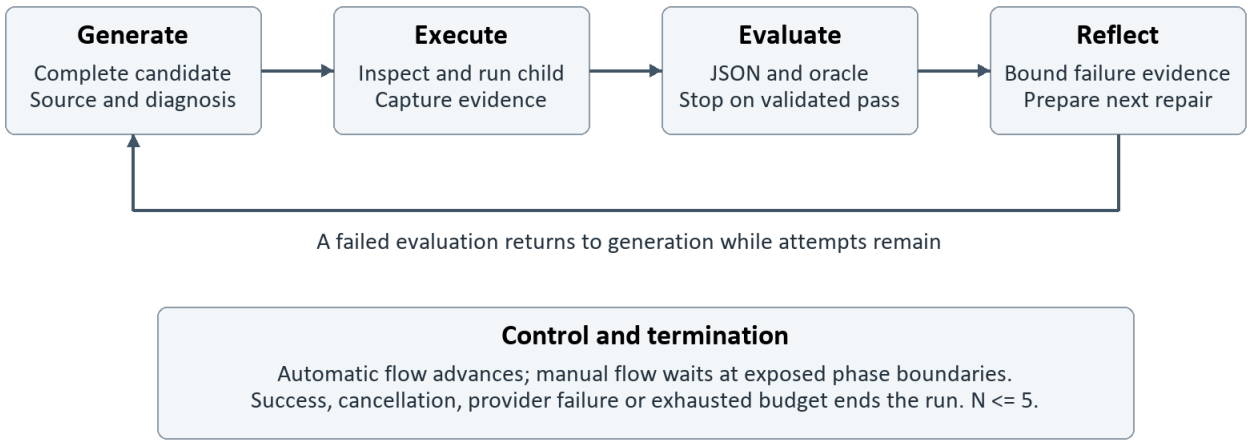


Figure 2 Shared execution loop with automatic progression and manual phase boundaries

Both flows use the same operations. Automatic flow advances without a human gate. Manual flow pauses so the presenter can inspect evidence before the next phase. Pacing changes; the validator, attempt budget and execution restrictions remain the same.

Phase	Observable work	Next decision
Generate	Record a complete candidate and concise technical diagnosis	Proceed to guarded execution
Execute	Capture stdout, stderr, return code, error and duration	Evaluate the actual output or failure
Evaluate	Check valid JSON and the selected contract	Stop on pass or prepare feedback
Reflect	Localize the defect and bound failure evidence	Request a repair if attempts remain
Done	Record success, exhaustion, cancellation or provider failure	No further candidate may execute

Termination and cancellation

Success stops the loop immediately; exhaustion preserves failure at the selected limit. Cancellation updates the visible state promptly and prevents delayed output from changing it. A cancelled worker retains its capacity slot until cleanup, preventing unlimited background work.

Manual runs pause BEFORE Generate, Execute, Evaluate and Reflect. The snapshot exposes the pending phase, attempt, sequence and one-use step_token. POST /api/runs/{id}/advance submits that token. A consumed token is idempotent and cannot advance the next phase; a conflicting token returns HTTP 409. Success ends directly after evaluation; cancellation wakes a waiting gate.

Modules API and Trace Records

Module	Responsibility
server.py	Serve the dashboard, validate local HTTP access, accept run actions and expose trace snapshots
engine.py	Manage state, attempts, phase progression, manual gates, capacity and termination
provider.py	Build bounded requests, call the selected model transport and normalize candidates
executor.py	Inspect source, launch the child, enforce limits and capture structured execution evidence
presets.py	Define public datasets, task rules and independently checked reference outputs
web directory	Render retro controls, source, logs, validation and trace navigation
tests directory	Exercise reference correctness, limits, provider behavior, state transitions and HTTP protections

Run and attempt records

A run records task, provider, control_mode, budget, status, input data and stage_history. Each stage has a sequence, phase, attempt, status and ready/start/finish times. Metrics distinguish generation, execution and manual waiting time. Attempts include source, reflection, raw generation_output, streams, return code, error, validation and exec_steps from real child observations.

HTTP contract

The API exposes service status and scenarios, starts a run, returns a run snapshot, supports its manual continuation and cancellation actions, and exports a trace. The exact routes and configuration fields are documented alongside the code. JSON mutations are accepted only for the local dashboard host and permitted origin. The service binds to IPv4 loopback rather than a public interface.

```
GET  /api/status and /api/presets
POST /api/runs
GET  /api/runs/{id}
POST /api/runs/{id}/advance  {"step_token": current_token}
POST /api/runs/{id}/cancel
GET  /api/runs/{id}/export
```

Trace history is bounded to forty runs and two unfinished workers. Unfinished cancelled workers cannot be evicted until cleanup completes. Exports contain task-related source and data-derived output; access logs do not need to contain these values. A trace is evidence for the recorded run, not a persistent memory that changes model weights.

Provider Requests and Bounded Feedback

Local and optional remote inference

The default provider sends chat requests to local Ollama with the selected coding model [4]. An optional API provider is a separate configuration choice [12]. Both must yield a complete Python candidate and a concise technical diagnosis in a structured response. Transport errors terminate the run explicitly. A provider's availability, latency or credentials must not change the execution policy or the independent oracle.

Request control	Value or contract	Purpose
Response wrapper	code and reflection strings	Separate executable source from a short public diagnosis
DATA contract	Native value; finite JSON up to 12000 characters	Prevent copied datasets and json.loads(DATA) mistakes
Local context	8192 tokens	Bound local inference while retaining repair context
Generation	Up to 1600 new tokens	Bound candidate generation; transport response is also capped
Previous source	Latest candidate up to 12000 characters	Preserve the latest repair source without full history growth
Feedback	Condense when encoding exceeds 6500 characters	Reduce streams and retain up to six mismatches when compacting
Feedback streams	stdout 1200; stderr 1800 characters	Retain precise evidence and reduce repetitive noise
Transport	30-second connection/read timeout; 180-second deadline watcher	Bound provider work independently of child execution

Repair exchange

The request preserves chat roles: original task and DATA are a user message, the previous incorrect candidate is an assistant message, and failure evidence is the next user message. Streaming fragments populate generation_output as received. The optional compatible API supports streamed content or a bounded ordinary JSON response. Transport diagnostics redact the configured credential. Expected-answer objects remain excluded from repair context.

```
TASK and DATA -> initial user message
PREVIOUS CANDIDATE -> assistant message
ERROR and VALIDATION EVIDENCE -> next user message
REPAIR -> new complete code and concise diagnosis
```

Field-specific feedback can say that highest must maximize an unrounded weighted score, or that every rejected row must be counted. This guides computation without giving the correct name or numeric total. Bounded feedback is an engineering compromise: it reduces context growth but may discard details needed by a difficult task.

The optional API path is covered by automated transport tests; no measured live external API result is claimed here.

Execution Controls and Safety Boundary

Generated source is parsed before execution. The constrained child receives DATA, approved builtins and a narrow collection of computation modules. It runs with isolated Python startup flags in a fresh temporary directory, without stdin and without inherited provider credentials. An audit hook adds a further guard against file, network, process and native operations. Subprocess execution provides stream capture and return-code handling [3]; it does not by itself create an OS security sandbox.

Control	Configured boundary	Observed behavior
Attempts	Integer from one to five	The loop stops at validated success or the selected limit
Execution time	One to five seconds	A hanging child is killed and a timeout is recorded
Generated source	24000 characters	Oversized or unsupported source is rejected before running
Captured output	16000-character display budget	Flooding is bounded and repeated adjacent lines are condensed
Imports	json, math, statistics, collections and re subset	Unapproved modules and exports are denied
Windows process	Best-effort 384 MB Job Object; one process	job_limit_active records whether the control attached
JSON output	One value, unique keys and finite numbers	Duplicate keys, NaN and exponent overflow fail explicitly

Resource and cancellation behavior

The parent drains stdout and stderr while enforcing cancellation, output and time limits. A runtime exception is recorded from the child rather than raised through the dashboard. Closing a Windows Job Object also supports child lifetime enforcement [7]. Its activation is reported per attempt because a best-effort platform control should not be assumed to have worked.

Appropriate deployment boundary

These controls support trusted local demonstrations. They do not fully contain hostile Python. Public deployment would require a stronger operating-system boundary with separate privileges, disabled network, filesystem restrictions and enforced resources.

Local previews include at most twelve names, shallow containers and bounded strings. DATA, modules, functions and private names are excluded; sensitive names are redacted. Tracing is optional and observer failures must not change the program result.

Child-side sys.settrace records lines before they execute and records raised exceptions [9]. Exceptions may be caught; error, return code and validation determine attempt failure. The parent separates nonce-prefixed records from stderr. The cap is 200 events and 24000 bytes per attempt, with trace_truncated reported. These are recorded CPython observations, rather than an interactive operating-system debugger.

Datasets and Task Algorithms

Sales cleaning and aggregation

Eight public rows contain numeric strings, empty and N/A values, and a negative adjustment. The algorithm strips each amount, attempts conversion, counts every accepted or rejected row, and aggregates valid values by region. Negative adjustments remain valid. The highest regional aggregate wins; alphabetical order applies only when the greatest totals tie.

```
total = 520.00
valid_rows = 6     rejected_rows = 2
by_region = {"North": 160.00, "South": 180.00, "West": 180.00}
top_region = "South"
```

Normalized text analytics

Three documents are tokenized using contiguous ASCII letters and lowercased. The algorithm counts all occurrences and unique tokens, then ranks by descending count and ascending token for ties. Punctuation is a separator, not a token. This deterministic rule makes the oracle reproducible without a language model.

```
document_count = 3     token_count = 14     unique_tokens = 7
frequencies = {"ai": 3, "heals": 1, "code": 4, "helps": 1,
               "reliable": 3, "results": 1, "tests": 1}
top_tokens = [
    {"token": "code", "count": 4},
    {"token": "ai", "count": 3},
    {"token": "reliable", "count": 3},
]
```

Weighted grade evaluation

Four students have three scores weighted 0.2, 0.3 and 0.5. The algorithm computes unrounded weighted values, rounds displayed averages to two decimals, averages the unrounded values for the class result, and classifies scores of at least 60 as passing. The highest name maximizes the unrounded score; alphabetical order is only a tie-breaker.

Student	Weighted average	Passes
Asha	78.00	Yes
Kabir	77.00	Yes
Mira	95.60	Yes
Dev	50.50	No

The class average is 75.28 and the highest student is Mira. These values document the public oracle. They are shown to the supervisor and validator, but are not injected as reference-answer objects into model repair prompts.

Validation Oracles and Metrics

Execution success and semantic success

Exit code zero means the process completed normally. It does not prove that the program followed a weighting rule or selected the greatest value. A preset therefore requires both executable output and agreement with an independent reference. The validator checks exact key structure, stable list order, value types and finite numeric values with a small floating-point tolerance. Booleans do not count as numeric answers.

Independent reference path

Reference tests recalculate the targets separately from the candidate programs. Sales and grade arithmetic use Decimal-based calculations, and text references use a separate counting path. This reduces the risk that a bug copied into both the candidate and its test will pass unnoticed. It does not establish coverage beyond the three fixed datasets.

Metric	Definition	Interpretation
Validated completion	Run ends with a passing preset oracle	Correct for the fixed contract and dataset
Attempt count	Number of candidate slots used	Includes a declared seeded first candidate where enabled
Repair count	Candidates after the first attempt	Does not prove each changed the source usefully
Execution duration	Time spent checking and running one candidate	Separate from model generation latency
Run duration	Elapsed time from creation to terminal state	Manual waiting may be included
Exhaustion	No candidate passed within the budget	A visible failure, not an application crash
Custom structural pass	A non-empty JSON object is received	No independent domain correctness claim

JSON is part of the contract

The child must print exactly one JSON value. Debug prints, multiple objects, duplicate keys and non-finite numbers are rejected [8]. Numeric exponent overflow such as 1e309 must also fail rather than becoming Infinity inside a trace. This keeps both the validator and exported JSON consistent.

Custom tasks accept finite, serializable JSON DATA up to 12000 ASCII-serialized characters. Verified preset data is fixed. The policy rejects rebinding or deleting DATA, including assignment through loop targets, comprehensions, function parameters and import aliases. Reading it through data = DATA remains allowed. None of these input checks supplies a custom semantic oracle.

The evaluation sample reports each observed run individually. Aggregate passes are descriptive only: a small fixed suite with deterministic decoding is not a statistically representative estimate of programming reliability. Scripted replay measures orchestration and execution rather than model intelligence.

Application Overview

The Win95-inspired interface presents the project as a small desktop workbench inside the browser. Its retro appearance is a presentation choice; task selection, provider identity, flow, limits and run state remain explicit. The following views document the running system rather than mockups.

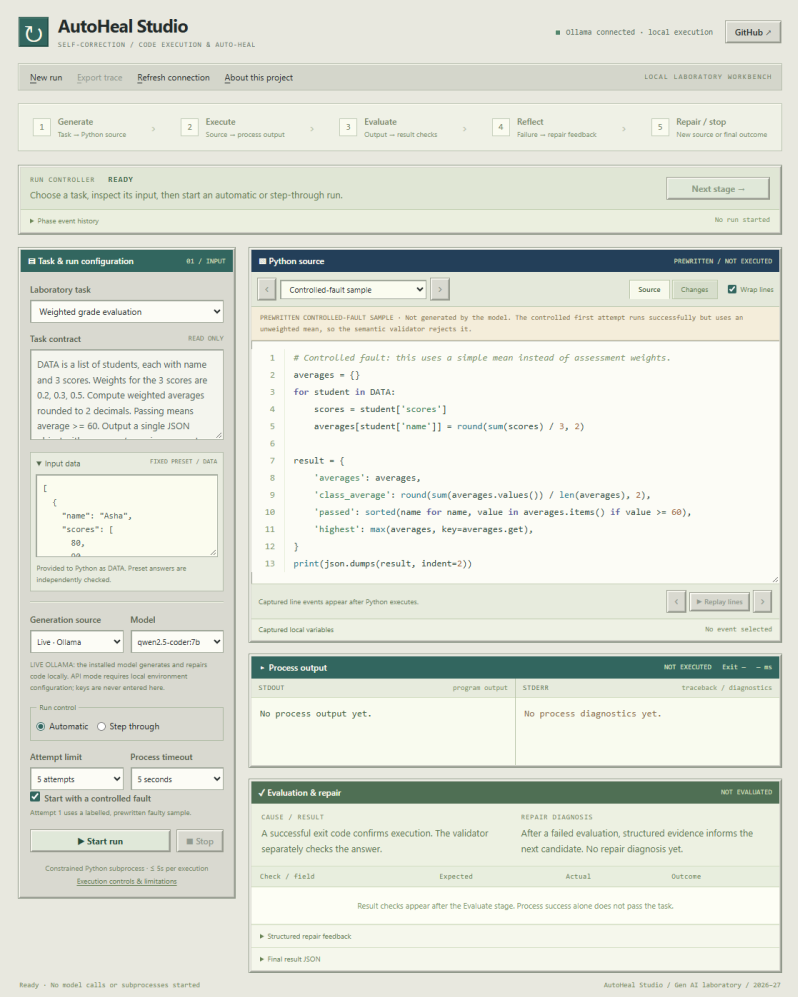


Figure 3 The complete Windows 95-inspired workbench, ready before a run.

Task contract, input DATA, provider and control settings, labelled prewritten Python sample, process output and result-check panels are visible. No model call or execution has occurred.

What the supervisor should inspect

Confirm the selected task, whether inference is live or scripted, and whether a controlled first failure is enabled. Then identify the attempt budget and execution timeout. The mode and provider labels explain what the next run can prove. An offline replay is useful for a repeatable walkthrough, while live cases provide separate evidence about model-generated repairs.

The source, execution, validation and trace views share one run identity. A viewer should be able to follow that identity from the controls to the exported record. This connection makes the demonstration reviewable after the lab session rather than dependent on a disappearing terminal window.

Manual Flow Candidate Inspection

Manual flow allows the presenter to inspect the candidate at a phase boundary before the next operation proceeds. The displayed source is the program that the executor will receive. A diagnosis is explanatory text; it is not itself executed and does not establish that the program is correct.

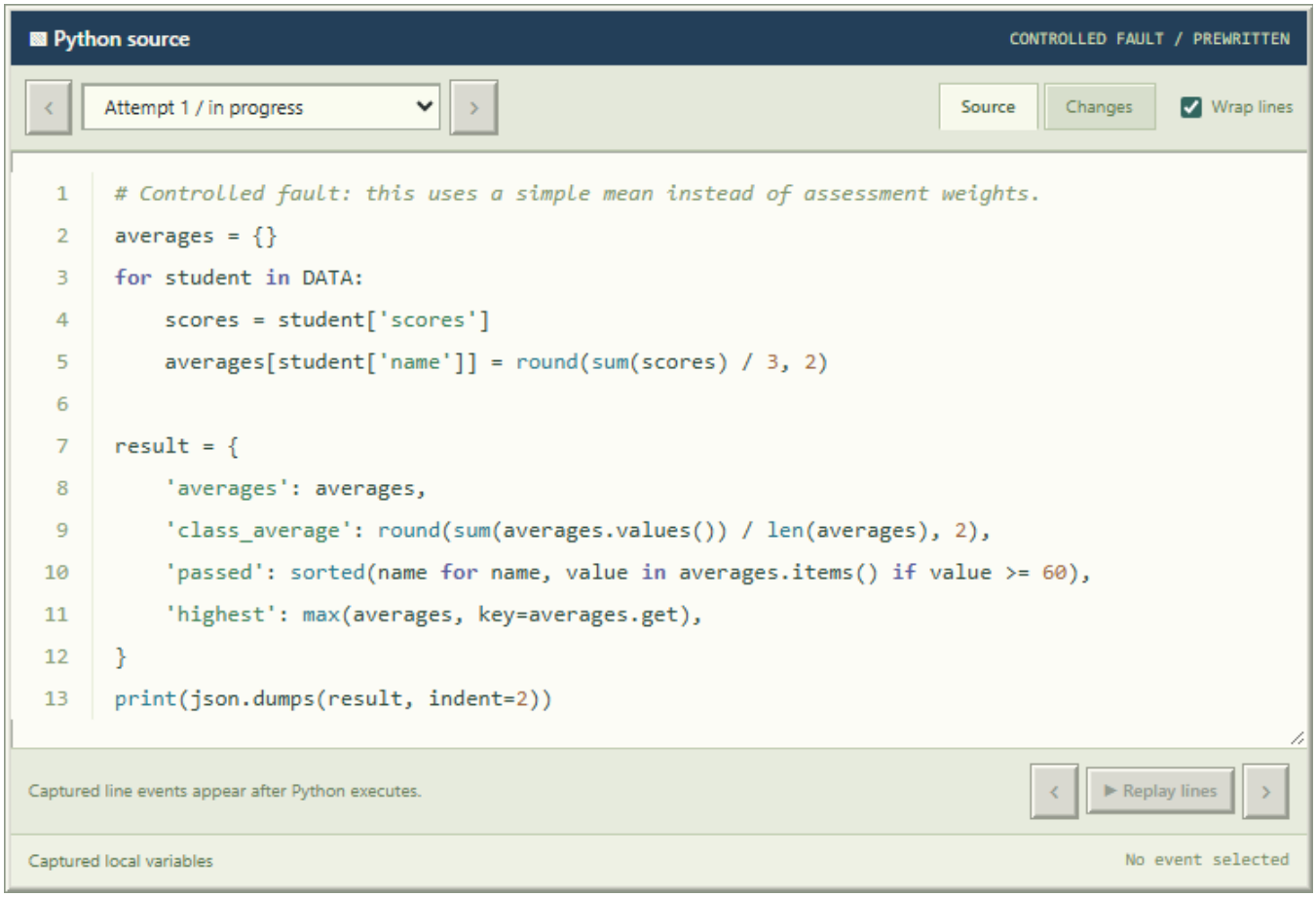


Figure 4 Step-through mode: the labelled controlled-fault candidate is ready before execution.

Grade replay attempt 1 is prewritten. The source calculates an unweighted average and has not executed at this paused boundary.

Evidence run da983fe180444321 | Mode replay

Controlled first failures

A declared first failure deliberately substitutes a known broken candidate to guarantee an explainable initial observation. The dashboard and trace identify this substitution. In a live seeded run, the model still has to generate the repair. In scripted replay, both candidates are predetermined and only the orchestration and execution are being demonstrated.

Before continuing, explain the defect visible in source and predict its consequence. The following views distinguish a program that runs but returns wrong values from a separate runtime exception. This makes the presentation an examination of implementation rather than unexplained button presses.

Semantic Failure After Successful Execution

The first grades replay candidate executes normally and prints a JSON object, yet its unweighted arithmetic violates the specified task. The independent evaluator rejects the result. This is the central distinction between a program that completes and an answer that is correct.

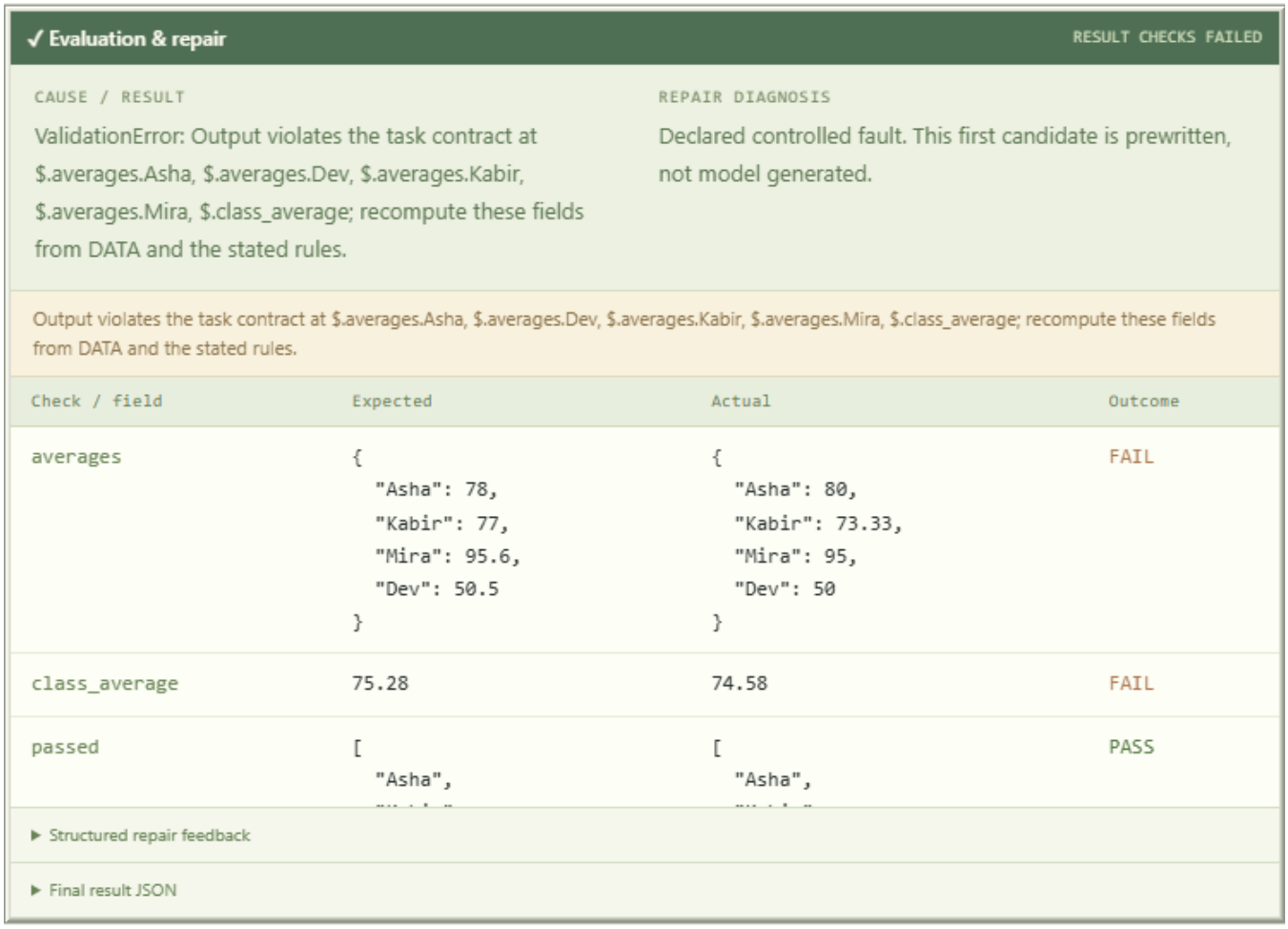


Figure 5 A zero exit code can still produce an incorrect answer.

Grade attempt 1 ran successfully but failed independent weighted-result checks; field-level expected and actual values are shown.

Evidence run da983fe180444321 | Mode replay

What the failed decision establishes

The mismatch paths identify affected output fields. The interface may show reference values to the supervisor for inspection, while repair feedback carries the violated rules and observed values without the expected-answer object. The evaluator remains separate from the candidate and does not accept an explanatory claim in place of correct computation.

This view is a labelled scripted replay, so it proves execution, independent rejection and observable orchestration. It does not claim that a model generated this first program or its predetermined replay repair. The later live evidence records actual Ollama-generated source separately.

Execution Failure and Runtime Observations

The sales replay records a real TypeError when the initial source adds raw string amounts. Stderr, exception type and the generated source line identify the failed operation. Stdout and stderr remain separate; a failed program may still produce partial output.

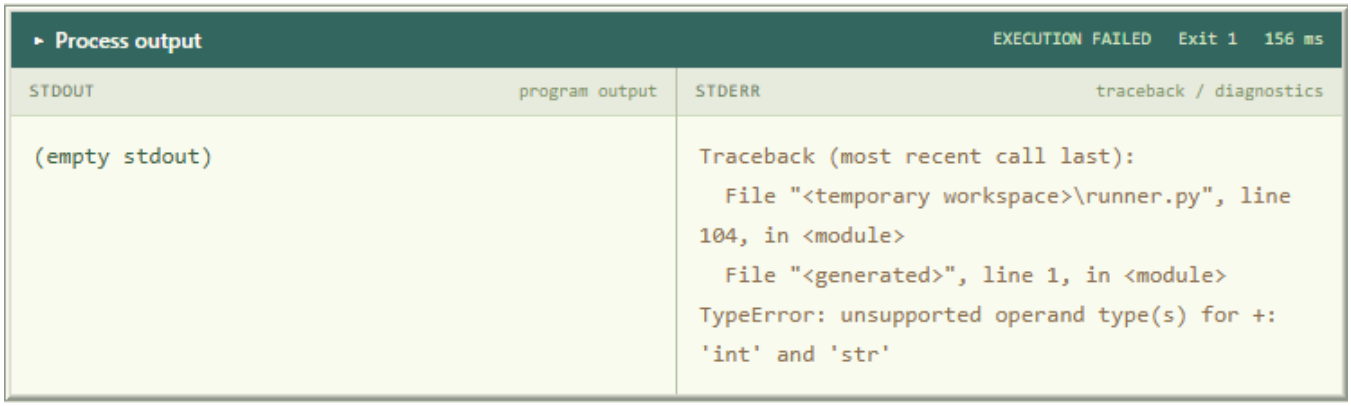


Figure 6 A separate sales replay demonstrates a real Python exception and traceback.

Attempt 1 sums raw string amounts using sum, whose starting value is numeric zero. Python exits with a TypeError; stdout, stderr, exit code and elapsed process time are captured.

Evidence run ba838b8a638341ef | Mode replay

Recorded local state

The second view is from the separate grades replay, identified below. A CPython line event contains local state before that line executes; an assignment can appear at the following event. Exception events record raised exceptions, including caught ones. Error, return code and validation determine whether the attempt failed.

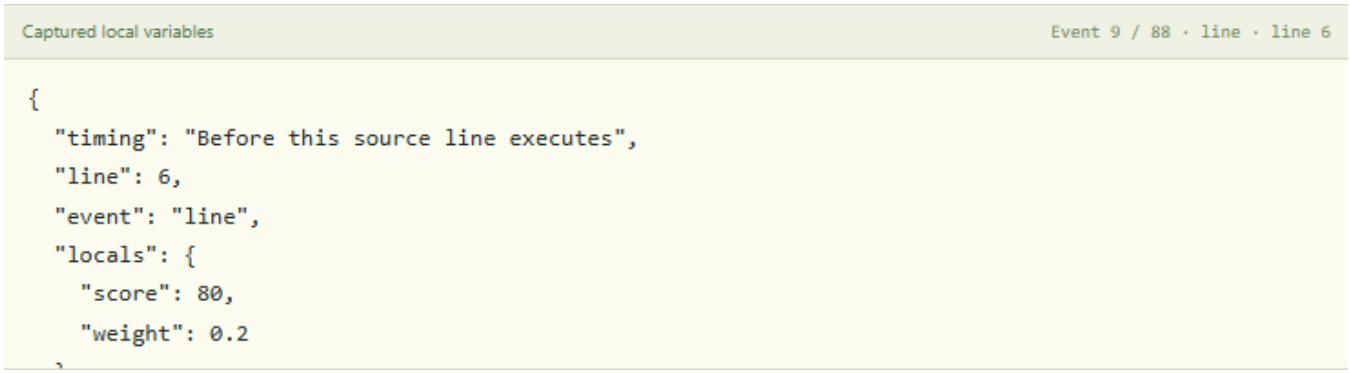


Figure 7 Captured Python line events show source location and bounded local-variable snapshots.

These are actual CPython observations recorded before each selected line, replayed after execution. They are not a live line debugger.

Evidence run da983fe180444321 | Mode replay

The interface replays recorded observations for inspection; it is not an interactive operating-system debugger. Bounded and redacted local snapshots aid explanation but do not capture every object. The complete candidate and run export provide the surrounding context.

Reflection and Structured Feedback

After a failed evaluation, the project converts the observation into a bounded repair record. It retains the latest candidate, exact error where available and localized validation failures. Repetitive log lines are condensed, long streams are truncated and temporary workspace paths are sanitized.

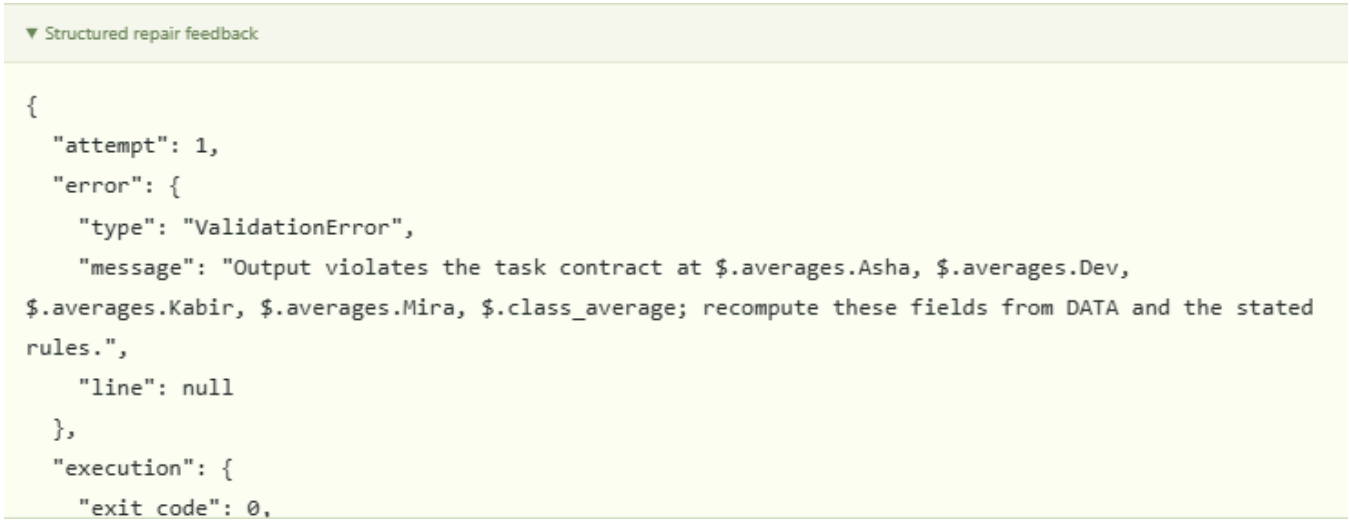


Figure 8 Structured failure evidence is ready for the next candidate.

The completed Reflect stage stores actual wrong result fields and corrective constraints. This scripted run uses fixed replacement source and makes no AI call. The scrollable JSON view is shown at its start.

Evidence run da983fe180444321 | Mode replay

Why semantic feedback names fields

A program can run successfully while choosing the alphabetically first name instead of the highest score. A field path such as $.highest identifies the failed contract. Its rule reminder tells the model to maximize the unrounded weighted score and use alphabetical order only for a greatest-score tie. It does not supply the expected name.

The displayed reflection is a short technical diagnosis. The system does not require hidden chain-of-thought, and the diagnosis is not accepted as proof of a repair. The next candidate must change the actual source and will undergo the same constrained execution and independent validation.

Repaired Candidate and Source Change

This complete replay repair uses the same task and DATA. Inspect its actual operations; an accompanying diagnosis is not proof. The source must compute the answer rather than print a known target, and it remains untrusted until execution and independent validation pass.

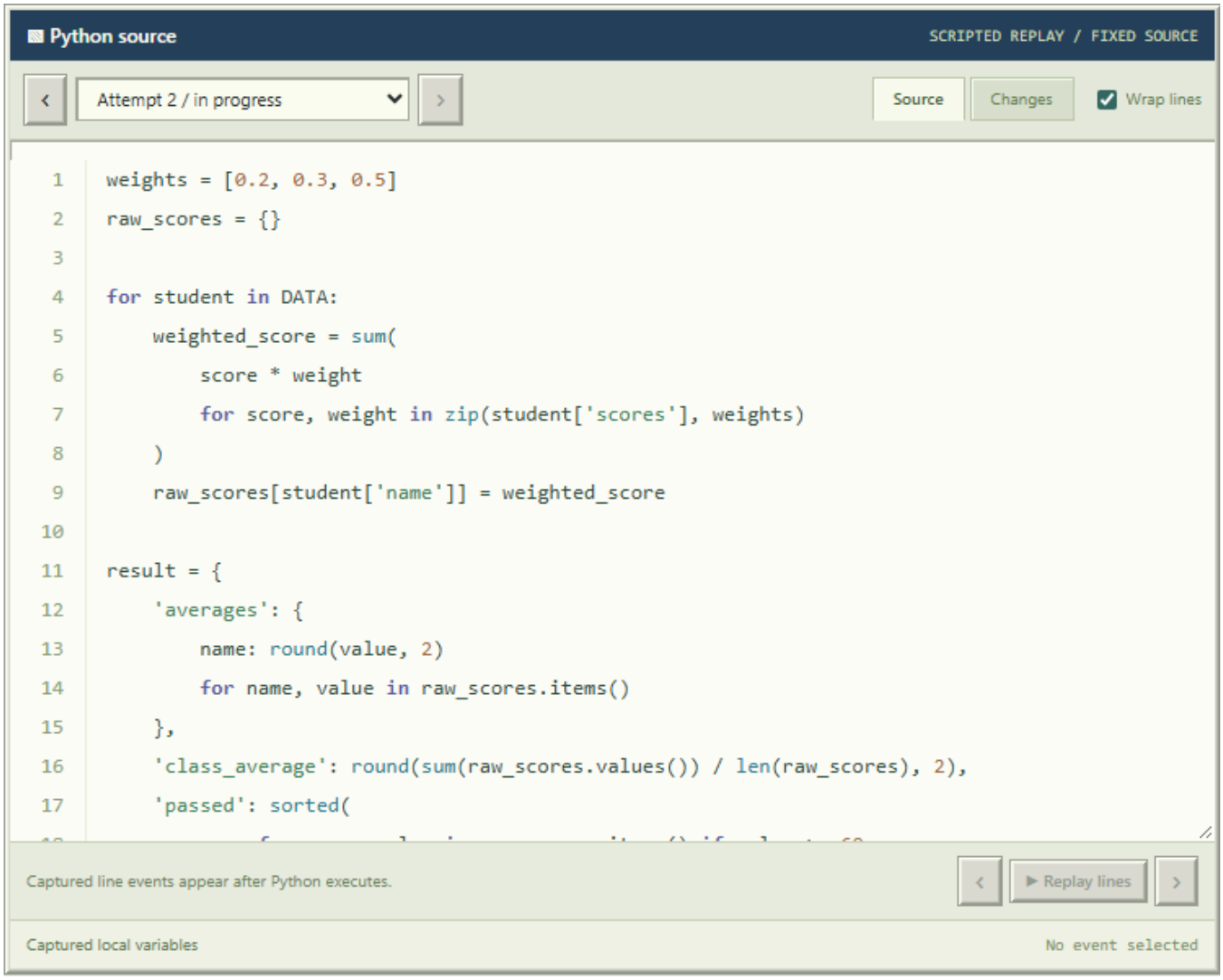


Figure 9 The second scripted candidate uses weighted scores and explicit result construction.

The replacement is fixed replay source, paused before Python execution. It uses the original DATA and does not substitute a reference answer.

Evidence run da983fe180444321 | Mode replay

How to explain the correction

The grades repair multiplies each score by its assessment weight, retains unrounded values for the class mean and highest selection, and rounds only displayed results. Sorting by descending score and then name implements the specified tie-break.

Source Diff and Repair Evidence

The diff compares stored consecutive candidates using a longest common subsequence of source lines. Insertions and removals are calculated from actual source; the display does not invent replacement lines.

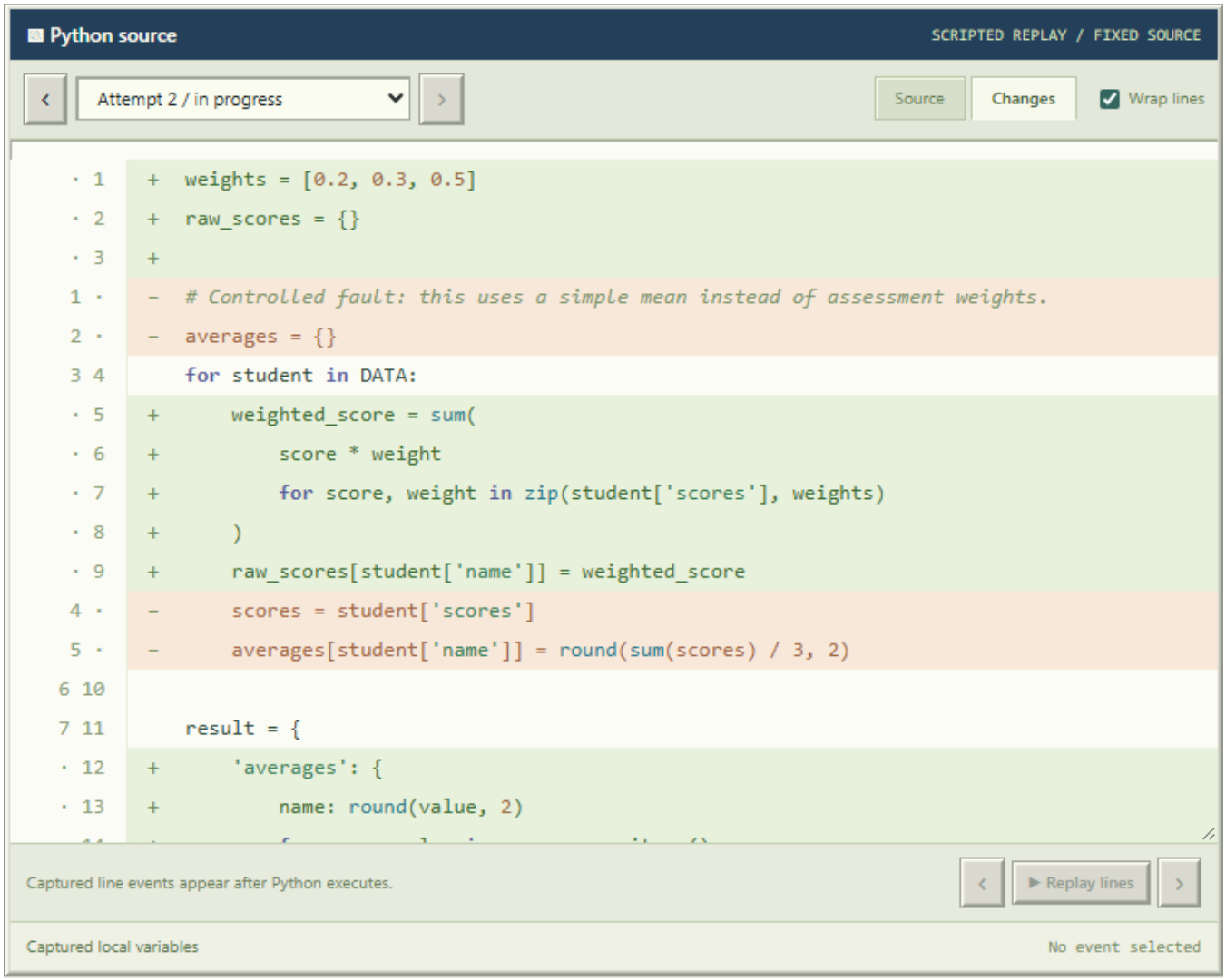


Figure 10 The Changes view compares the actual first and second candidate sources.

Added and removed lines are computed from stored source using an LCS comparison; code remains selectable with true source line numbers.

Evidence run da983fe180444321 | Mode replay

Relating the edit to the observed defect

Inspect the changed grade aggregation and selection rules against the original semantic failure. A source edit alone does not prove correct arithmetic; the next execution and independent validation establish that. Unchanged source is also visible, making a non-converging repair explicit.

Independent Validation and Termination

The validation view compares the actual result with the selected contract. For a verified scenario, a passing decision requires the complete expected structure and values. A failed semantic decision remains a failure even when the process returned zero. The terminal run status records what the loop established.

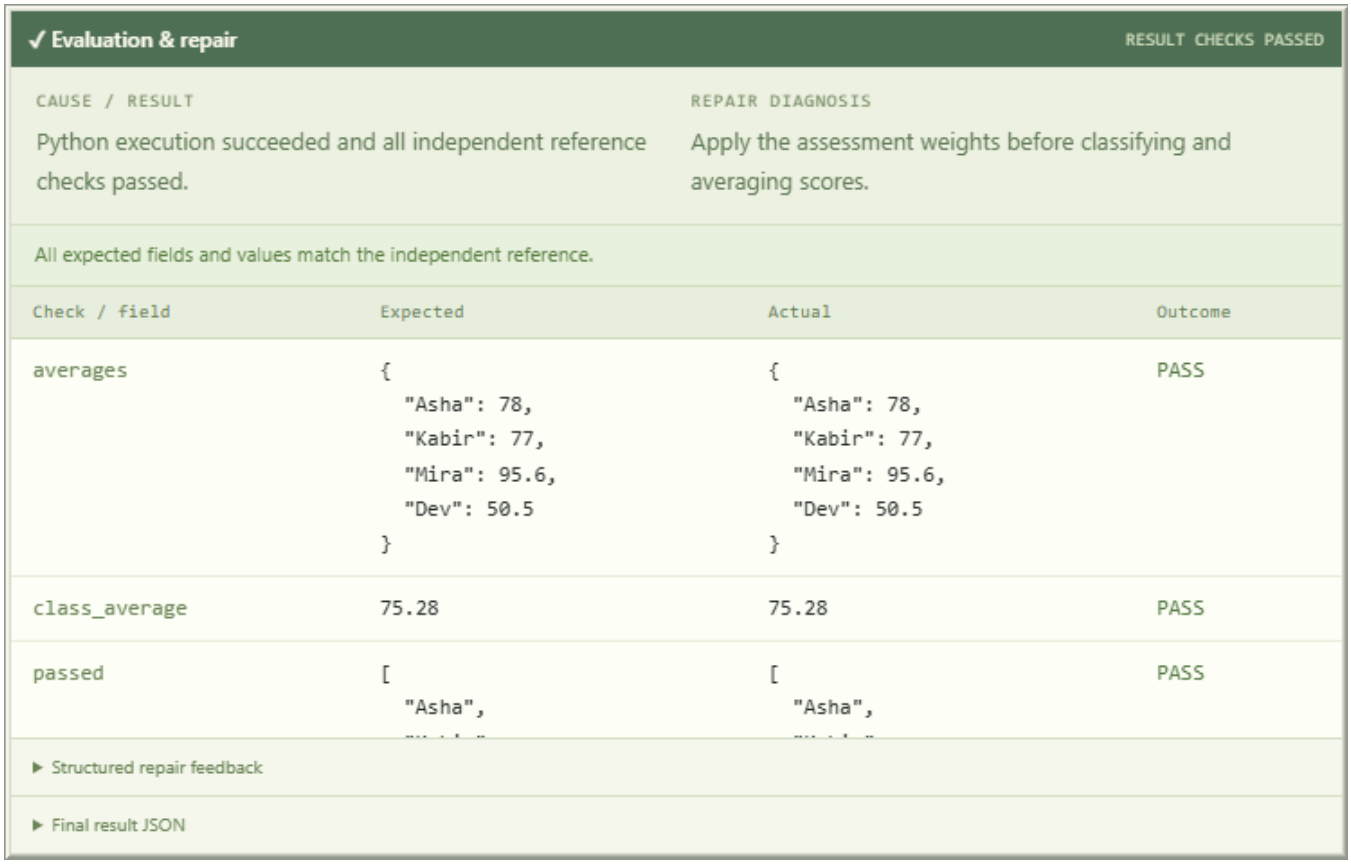


Figure 11 Grade replay terminates after the independent reference checks pass.

All fixed-preset result fields pass and the loop stops at attempt 2. Final output is available in the JSON disclosure and export.

Evidence run da983fe180444321 | Mode replay

Interpreting the final state

Succeeded means the selected validation passed and no further attempt was launched. Exhausted means the permitted candidates failed. Cancelled preserves the user's stop request against late provider output. Provider error preserves a transport or configuration failure without synthesizing a successful answer. The trace retains earlier attempts so a final pass does not erase its failure history.

For a custom task, the interface explicitly reports structural checking. A non-empty JSON object may still contain a wrong domain answer. A supervisor should read the validation kind as well as the success label before drawing a conclusion about correctness.

Live Model Repair Evidence

This separate manual run uses the local Ollama model for the repair. Unlike the predetermined walkthrough, the displayed candidate is the actual source returned by the provider. The run is paused before execution so the supervisor can inspect the complete program and the concise public diagnosis.

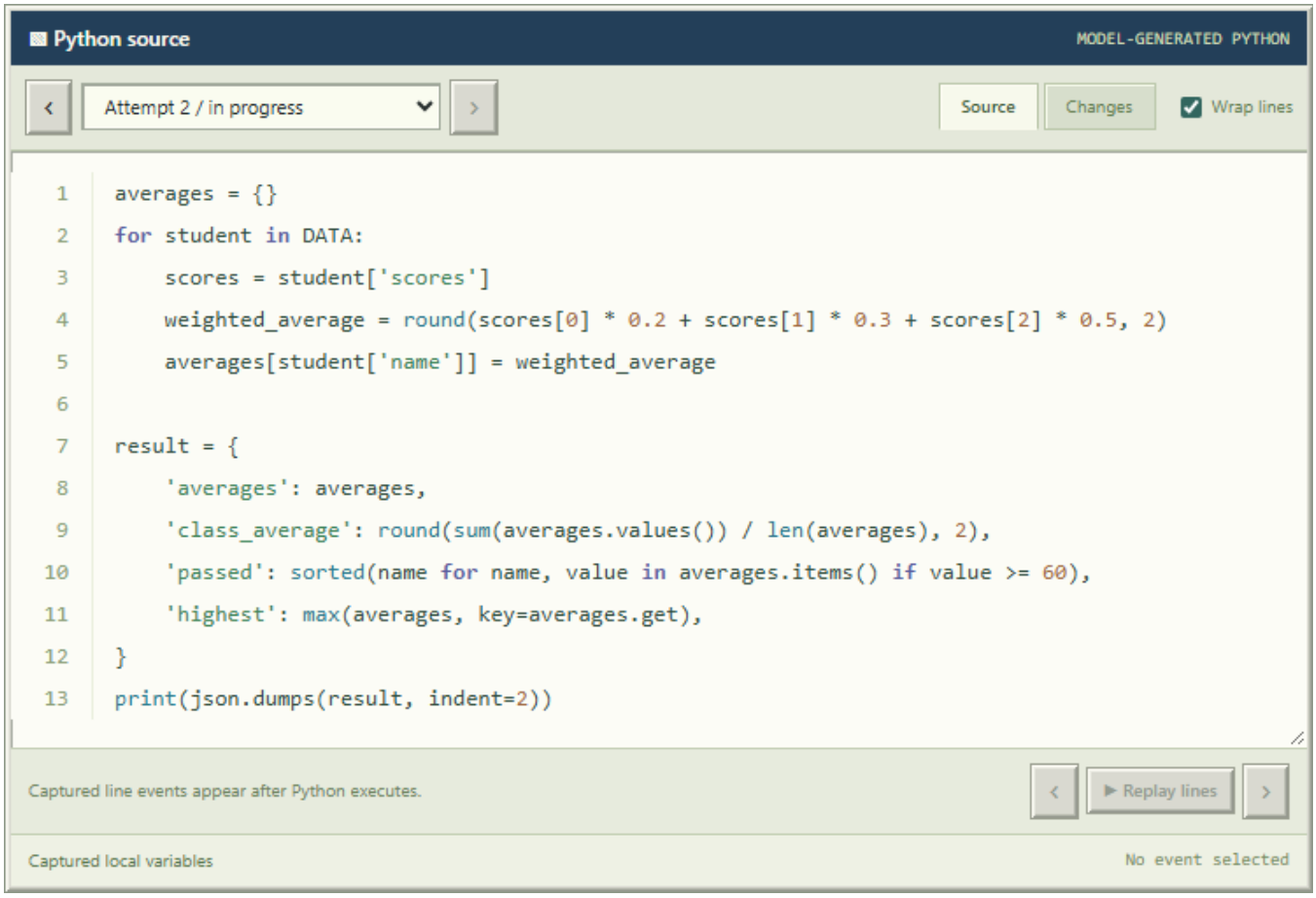


Figure 12 A live Ollama repair is paused before execution in the same step-through interface.

Unlike the scripted walkthrough, this candidate was actually generated by the selected local model from the controlled grade fault and its failed result checks. The source header identifies it as model-generated Python. The captured state precedes its execution.

Evidence run c7c5aaf24c154d32 | Mode live

Generation is an observable intermediate result

The source and raw stream establish what the model proposed. This run later passed the fixed grade oracle on attempt two, as its saved trace records. However, the generated class mean averages rounded individual values; unseen rounding-edge inputs remain unverified. A fixed-example pass therefore does not establish full generality. This manual run is separate from the six automatic evaluation cases and their latency measurements.

Automatic Progression and Trace Export

Automatic flow applies the same phases without presenter gates. The timeline exposes their sequence. The actual JSON export records the same run identity, candidates, execution and validation; review source and task-derived values before sharing.



Figure 13 Automatic replay advances through the recorded stages and stops after successful validation.

Text replay executes a failing source and a corrected source without manual stepping. The event history records actual Generate, Execute, Evaluate and Reflect stage transitions.

Evidence run 830e7cf09f5c4c7c | Mode replay

Pretty-print ☐


```
{
  "id": "da983fe180444321",
  "status": "succeeded",
  "phase": "done",
  "phase_status": "complete",
  "task": "DATA is a list of students, each with name and 3 scores. Weights for the 3 scores are 0.2, 0.3, 0.5.
Compute weighted averages rounded to 2 decimals. Passing means average >= 60. Output a single JSON object with
averages (mapping names to average), class_average (2 decimals, computed from unrounded individual averages),
passed (alphabetically sorted names), and highest (the name whose unrounded weighted average is greatest;
choose the alphabetically first name only when the greatest scores tie). DATA is already defined; compute all
outputs from it without redefining or copying the dataset into source.",
  "preset_id": "grades",
  "mode": "replay",
  "provider": "ollama",
  "model": "scripted-replay",
  "control_mode": "manual",
  "max_attempts": 5,
  "timeout": 5.0,
  "seed_failure": true,
  "trace_execution": true,
  "input_data": [
    {
      "name": "Asha",
      "scores": [
        80,
        90,
```


Figure 14 The exported run is machine-readable JSON with exact input, candidates, execution and stage evidence.

The dashboard Export trace button downloaded this same run; its ID was checked against the current run. The browser view shows the real API snapshot, with no manufactured fields.

Evidence run da983fe180444321 | Mode replay

Automated Verification

The current implementation passed 63 automated tests in 14.162 seconds in the recorded environment. The archived test log is docs/evidence/test-results.txt. This is a measured software verification result, separate from model coding benchmarks and live run pass rates.

```
python -m unittest discover -s autoheal/tests -v
```

Verification area	Material behavior checked
Independent references	Recalculate public oracle results through separate arithmetic and token-counting paths
Execution diagnostics	Capture exception type, exact generated line, stdout, stderr and return code
Runtime observations	Verify actual pre-line state, exception line, bounded redacted locals and truncation; disabling tracing and observer errors do not alter results
Resource bounds	Kill hanging children, bound flooding and reject unsupported access
JSON integrity	Reject malformed output, duplicates, multiple values, NaN and numeric exponent overflow
Loop termination	Stop on first validated success, cap attempts and preserve exhaustion or provider failure
Cancellation and capacity	Keep cancelled status authoritative and retain worker slots until cleanup
Manual control	Check all seven gates in a two-attempt run, conflicting tokens, idempotent duplicate advances and waiting cancellation
Provider transport	Check local and API request formats, streamed fragments, configured models, redirects and secret redaction
HTTP and export	Reject unapproved origins and hosts; preserve valid trace serialization

Why regression cases matter

The exponent-overflow regression rejects 1e309 before JSON export. Cancellation tests retain worker capacity until cleanup. A later diagnostic exposed Counter.most_common lazily importing heapq inside CPython. The trusted runner now preloads that dependency before installing its import guard; generated code remains forbidden from importing heapq. Typed shape feedback also distinguishes an expected object from an actual array without supplying expected answer values.

Automated tests can substitute a deterministic provider response to isolate orchestration behavior. Such tests establish the software contract they exercise; they do not establish that the installed language model will repair a new task. Live inference is evaluated separately on the next page.

The Windows and Linux GitHub Actions jobs also passed for implementation revision e5a064a. This cross-platform check verifies the test suite on both runners; it is separate from the laptop latency measurements.

GitHub Actions verification run 38933564801

Current Live Evaluation Results

The table records the current evaluation cases associated with this implementation revision. Each entry has a saved trace. Durations cover complete runs rather than only Python execution. Manual flow may include time spent waiting for human continuation and should not be compared directly with uninterrupted automatic latency.

Mode	Flow	Task	Attempts	Seconds	Outcome
Live seeded	auto	Sales	3	42.41	succeeded
Live seeded	auto	Grades	2	18.30	succeeded
Live seeded	auto	Text	3	23.88	succeeded
Live generated	auto	Sales	3	168.06	succeeded
Live generated	auto	Grades	2	44.03	succeeded
Live generated	auto	Text	1	18.56	succeeded

6 of 6 recorded live cases reached validated completion. This is a descriptive count for the fixed lab scenarios, not an estimate of general programming accuracy. Seeded live cases include a deliberately broken first candidate, while fully generated cases ask the model to produce every ordinary candidate.

Evidence and interpretation

The expected outputs are independently defined, public fixed examples. The sample is small, tasks are related, and repeated deterministic prompts are not independent draws from real software work. Passing can demonstrate the loop's mechanics without proving broad generalization. All six post-fix cases are reported here; historical failures remain in the diagnostic archive below.

A historical pre-fix diagnostic recorded 3 of 6 passing cases; the Counter lazy-import issue affected the text scenario. That record is preserved as docs/evidence/diagnostic-pre-fix-summary.json. The main table reports the separate rerun after the runner correction and typed shape diagnostics, rather than replacing history with a successful replay.

The optional API path is covered by automated transport tests; no measured live external API result is claimed here.

Recorded environment

Python 3.12.14. Windows laptop; Intel Core i7-13650HX; 23.71 GiB installed usable RAM reported by Windows; NVIDIA GeForce RTX 3050 6GB Laptop GPU. The selected model and provider are stored in each trace. These conditions matter when interpreting latency and reproducibility.

Failure Analysis and Non Convergence

Failure class	Example mechanism	Correct response
Syntax	A malformed expression cannot parse	Return exact source location for a later repair
Runtime	A raw string sum or nonexistent method raises an exception	Supply the concrete exception and generated line
Logical	A simple mean replaces a weighted mean, or min selects a highest name	Reject using the independent contract and identify failing fields
Output format	Extra prints, duplicate keys or non-finite numbers	Reject the contract violation before accepting a result
Policy or timeout	Unsupported access or a hanging computation	Keep the restriction; rework or terminate
Provider	Unavailable service, missing model or API error	Stop explicitly and preserve partial evidence
Exhaustion	Repeated candidates fail within the allowed budget	Stop and require a deliberate new run or human review

Why a plausible diagnosis may not converge

A repair can repeat the same expression while describing the correct rule in words. The model may focus on the traceback but miss a downstream semantic defect, or correct one field while introducing another mismatch. Bounded history reduces context growth but can also remove useful information. These mechanisms explain why more iterations alone are not a guarantee of correctness.

Interpreting syntax and logical errors

Syntax errors are often repairable and should not be classified as inherently unrecoverable. A logical error can be harder to detect because the program exits normally. In this project, a failure becomes non-recoverable within the current run when the attempt budget is exhausted, the provider fails, or the required computation lies outside the permitted environment. The terminal status makes that boundary explicit.

A useful supervisor variation

Change a task rule or data value, then explain what the validator and feedback would need to change. With a fixed preset, changing only the natural-language task while keeping the old oracle would create an invalid experiment. A generalization study should use matched, independently calculated unseen datasets rather than merely rerunning a memorized public example.

Supervisor Demonstration and Viva

Suggested demonstration sequence

Start with the selected provider and mode visible. Use manual flow to expose a declared failure, inspect the exact source and traceback, show the bounded feedback, then continue to the repaired candidate and validation. Explain whether the sequence uses live repair or scripted replay. Finish with an automatic run and open its exported JSON trace. Keep an observed failure available to explain exhaustion honestly.

Viva question	Concise explanation
Why is exit code zero insufficient?	It proves normal process completion, not compliance with the task. The preset oracle checks semantics.
Where does generated code run?	Inside a fresh constrained child Python process, not in the server or browser.
What is passed back to the model?	The latest candidate and bounded structured error or validation evidence, without reference answers.
Does the model learn new weights?	No. It receives a revised inference context; the project performs no model training.
Why offer manual and automatic flow?	Manual flow exposes phase boundaries for teaching. Automatic flow demonstrates uninterrupted orchestration.
What happens at five failed attempts?	The run ends as exhausted and no further candidate is executed.
What does replay prove?	Real orchestration, diagnostics, execution and validation, using predetermined source rather than model intelligence.
Why is the executor not an OS sandbox?	AST, builtins, audit hooks and subprocess limits are layered controls, but do not provide a complete hostile-code boundary.
What does custom success mean?	Only the advertised structural contract passed; the domain answer has no independent oracle.
Why can cancellation still retain a worker slot?	The UI stops immediately while underlying transport or process cleanup may still be finishing.

Both students should be able to locate the relevant module, explain one source change and reproduce an exported run. An accurate explanation of a failed live run is stronger evidence of understanding than presenting a scripted success as model-generated behavior.

Limitations Ethics and Future Work

Technical limitations

The approved Python subset excludes file analysis, network retrieval, external packages and several language features. A correct but slow candidate can hit a resource boundary. Recorded local-variable previews are deliberately shallow, capped and redacted. The fixed examples do not establish unseen-task reliability, and custom JSON has no general oracle. Published Qwen checkpoint scores are not measurements of this quantized application [5,6].

Privacy and provider choice

Local Ollama inference avoids sending task content to a hosted model during that path. Selecting an API provider changes the privacy boundary because task data, code and failure evidence may leave the computer. Keep credentials on the server side, exclude them from generated-child environments and repository history, and review exported traces before sharing. Use public teaching data rather than personal or confidential records in demonstrations.

Academic and responsible AI considerations

This project was prepared with AI-assisted implementation and writing. Both students should inspect the code, understand its failure mechanics and disclose assistance according to faculty policy. Generated code may contain bugs, unsafe patterns or uncertain attribution. Human review remains necessary before real deployment. The project makes no demographic fairness claim, security certification or guarantee of original output.

Prioritized future work

First, strengthen evaluation with unseen dataset variants and independently calculated oracles. Second, add a real operating-system sandbox before considering untrusted public workloads. Third, compare providers under the same tasks, settings and cost accounting, preserving unsuccessful runs. Further work could add test generation that remains independent of the candidate, richer source comparisons, accessibility checks and artifact persistence with explicit retention controls.

Improvement should be measured against a specified defect. A larger model or a more elaborate interface does not replace independent validation, careful resource boundaries and an auditable record of what actually ran.

Conclusion References and Reproducibility

AutoHeal integrates model proposals, constrained execution, concrete observations, independent validation and bounded repair in one demonstrable system. Manual flow exposes the mechanics; automatic flow applies the same controls without presenter gates. The evidence supports the behavior of the recorded revision and cases, including failures, rather than a claim of universal autonomous correctness.

Version 2.0.0 | Implementation revision e5a064a7f31ac2c452127edef850e8b99066be6f | 16 October 2026

https://github.com/aayushtiwari845/AutoHeal

Reproduce the software checks with the command in the automated verification section. The recorded test log is docs/evidence/test-results.txt. Current run records are indexed by docs/evidence/live-summary.json, and UI captures by docs/screenshots/index.json. These files connect the report to source and observed execution.

References

[1] Yao et al 2023. ReAct Synergizing Reasoning and Acting in Language Models.
https://arxiv.org/abs/2210.03629

[2] Shinn et al 2023. Reflexion Language Agents with Verbal Reinforcement Learning.
https://arxiv.org/abs/2303.11366

[3] Python Software Foundation. Subprocess management documentation.
https://docs.python.org/3/library/subprocess.html

[4] Ollama. Generate a chat message and structured responses.
https://docs.ollama.com/api/chat

[5] Qwen Team. Qwen2.5 Coder 7B Instruct official model card.
https://huggingface.co/Qwen/Qwen2.5-Coder-7B-Instruct

[6] Hui et al 2024. Qwen2.5 Coder Technical Report.
https://arxiv.org/abs/2409.12186

[7] Microsoft. Job Objects in the Windows process documentation.
https://learn.microsoft.com/en-us/windows/win32/procthread/job-objects

[8] Python Software Foundation. JSON encoder and decoder documentation.
https://docs.python.org/3/library/json.html

[9] Python Software Foundation. CPython tracing and sys settrace documentation.
https://docs.python.org/3/library/sys.html#sys.settrace

[10] Project source and evidence. AutoHeal repository.
https://github.com/aayushtiwari845/AutoHeal

[11] Ollama. Qwen2.5 Coder 7B quantized distribution.
https://ollama.com/library/qwen2.5-coder:7b

[12] Groq. Compatible chat API documentation.
https://console.groq.com/docs/openai

References accessed 16 October 2026. Assessment basis: supplied GenAI LAB CA scheme and experiment list for 2026–27.